

Compiler Design - Module 1

Basic Level Study Guide

For students beginning their study of Compiler Design

1. Phases of Compiler

What is a Compiler?

A **compiler** is a program that converts source code into target code. Compilation is done in different steps called **phases of compiler**.

The Six Main Phases:

Phase	Description
1. Lexical Analysis	Reads characters and groups them into tokens
2. Syntax Analysis	Checks grammar and creates syntax tree
3. Semantic Analysis	Checks meaning and variable types
4. Intermediate Code Generation	Converts to intermediate form like three-address code
5. Code Optimization	Improves code to make programs faster
6. Target Code Generation	Produces machine code or assembly code

1. Lexical Analysis (First Phase)

- Reads characters of the program
- Groups characters into tokens

Example:

```
position = initial + rate * 60
```

Tokens produced:

```
position , = , initial , + , rate , * , 60
```

2. Syntax Analysis

- Checks grammar of the program
- Creates a syntax tree

Example:

```
a = b + c
```

The compiler checks if the statement follows grammar rules.

3. Semantic Analysis

- Checks meaning of the program
- Checks types of variables

Example:

```
int a;  
a = "hello"; ■ Error (type mismatch)
```

4. Intermediate Code Generation

- Converts program to intermediate form
- Often uses three-address code

Example:

```
t1 = rate * 60  
t2 = initial + t1  
position = t2
```

5. Code Optimization

- Improves the code
- Makes the program faster and smaller

Example:

```
Before: t1 = rate * 60.0; position = initial + t1;  
After: position = initial + rate * 60.0;
```

6. Target Code Generation (Final Phase)

- Produces machine code or assembly code

Example:

```
MOV R1, rate  
MUL R1, 60  
ADD R1, initial  
STORE position
```

2. Tokens, Lexemes, and Patterns

These three concepts are used in **Lexical Analysis**.

What are Tokens, Lexemes, and Patterns?

Concept	Meaning	Example
Token	Logical unit with meaning	identifier, keyword, number
Lexeme	Actual text in the program	a, b, 10
Pattern	Rule for forming a token	letter + digits

Token (Logical Unit)

A token is a group of characters that has meaning in the program.

Types of tokens:

- Identifier (variable names)
- Keyword (reserved words like if, while)
- Number (digits)
- Operator (+, -, *, /)

Example:

```
Program: a = b + 10
```

```
Tokens: id , = , id , + , num
```

Lexeme (Actual Text)

A lexeme is the actual text or sequence of characters in the program.

Example:

```
Program: a = b + 10
```

```
Lexemes: a , = , b , + , 10
```

Pattern (Rule for Token)

A pattern is a rule that describes what characters can form a token.

Examples of patterns:

Token Type	Pattern
Identifier	letter (letter digit)*
Number	digit+
Keyword	if while else

Relational Op	< <= > >= ==
---------------	----------------------

3. Regular Expressions

Regular expressions are used to describe patterns for tokens.

What is a Regular Expression?

A regular expression is a pattern that describes a set of strings. They are used for **specification of tokens** in lexical analysis.

Basic Symbols

Symbol	Meaning	Example
a	A single character	a matches 'a'
a*	Zero or more 'a'	a* matches '', 'a', 'aa', 'aaa'
a+	One or more 'a'	a+ matches 'a', 'aa', 'aaa'
a b	Either 'a' or 'b'	(if else) matches 'if' or 'else'
ab	Concatenation of 'a' then 'b'	bab matches 'ab'

Examples of Regular Expressions

1. Identifier:

```
letter (letter | digit)*
```

Matches: a, abc, var1, xyz123

2. Number:

```
digit+
```

Matches: 1, 123, 9999

3. Relational Operator:

```
< | <= | > | >= | ==
```

Matches: <, <=, >, >=, ==

4. Keyword:

```
if | else | while | for
```

Matches: if, else, while, for

Uses of Regular Expressions

Regular expressions are used to build:

- DFA (Deterministic Finite Automata)
- NFA (Non-Deterministic Finite Automata)
- Lexical analyzers

4. Symbol Table

What is a Symbol Table?

A symbol table is a data structure used by the compiler to store information about identifiers (variables, functions, constants) used in the program.

Information Stored in Symbol Table

Information	Description	Example
Name	Name of identifier	a, b, count
Type	Data type	int, float, char
Scope	Where it is accessible	local, global
Address	Memory location	100, 200, 300

Example: Symbol Table

Program code:

```
int a;  
float b;
```

Symbol Table created:

Name	Type	Scope	Address
a	int	local	100
b	float	global	200

Uses of Symbol Table

- Store variables and their information
- Store functions and their parameters
- Store constants
- Used during semantic analysis (type checking)
- Used during code generation (to find memory addresses)

5. DFA and NFA (Finite Automata)

What are Finite Automata?

Finite automata are abstract machines used in lexical analysis to recognize tokens. They are used to convert regular expressions into automata that can scan and identify tokens in the source code.

NFA - Non-Deterministic Finite Automata

Properties:

- Can have multiple transitions from one state on the same input
- Epsilon (ϵ) moves are allowed (transitions without consuming input)
- Easy to design

Example: NFA for pattern 'ab'

```
q0 --a--> q1 --b--> q2
```

The NFA has states q0, q1, q2. From q0, on input 'a' it goes to q1. From q1, on input 'b' it goes to q2 (final state).

DFA - Deterministic Finite Automata

Properties:

- Only one transition from each state on a given input
- No epsilon (ϵ) moves
- Faster to execute
- Used in actual lexical analyzers

Example: DFA for Identifier

```
q0 --letter--> q1
```

```
q1 --letter/digit--> q1
```

From state q0, on input of a letter it goes to q1 (final state). From q1, on input of a letter or digit, it stays in q1.

NFA vs DFA Comparison

Aspect	NFA	DFA
Transitions	Multiple transitions allowed	Single transition per symbol
Epsilon Moves	Allowed (ϵ)	Not allowed
Design	Easy to build	More complex to build
Execution	Slower	Faster
Use in Compiler	For specification	For actual implementation

This is a basic-level study guide for Module 1 - Compiler Design. For advanced topics and intermediate explanations, consult your course materials.